

操作系统，2026

第4讲 线程及其实现

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

线程的基本概念

02

线程的实现机制

03

Linux和Windows的线程

引入线程的动机

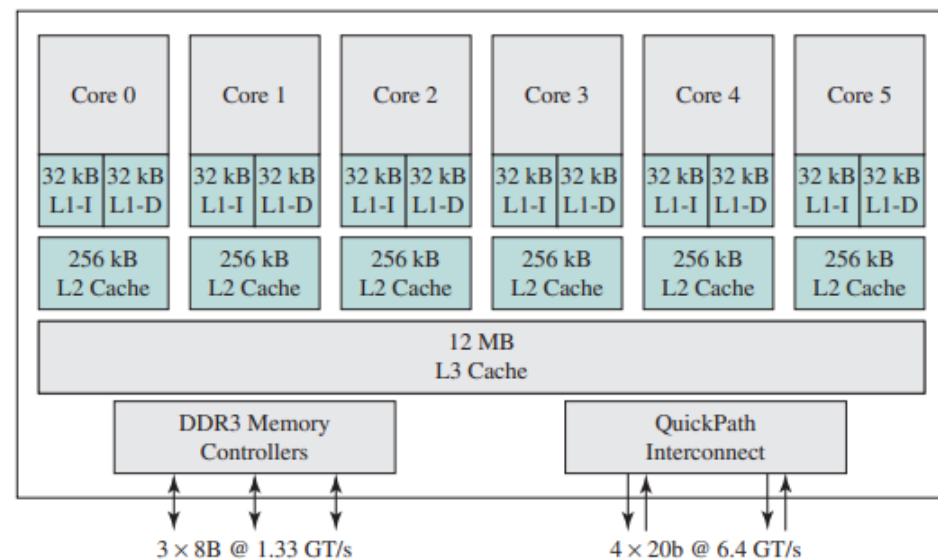
- 在操作系统中引入进程是为了使多个程序能够并发执行，提高资源利用率和系统吞吐量
- 进程的两个基本属性：
 - **资源分配单位**：每个进程拥有虚拟地址空间，保存进程映像，控制一些资源（例如文件、I/O设备），有状态、优先级
 - **CPU调度单位**：进程是一个程序的执行轨迹
- 以上两个属性构成进程并发执行的基础

引入线程的动机1

- 由于进程是资源分配单位，因而在创建、终止、切换的过程中，系统必须为之付出较大的时空开销
- 因此，系统中并发执行的进程的数目不宜过多，进程切换频率也不宜过高，这样就限制了系统的并发程度

引入线程的动机2

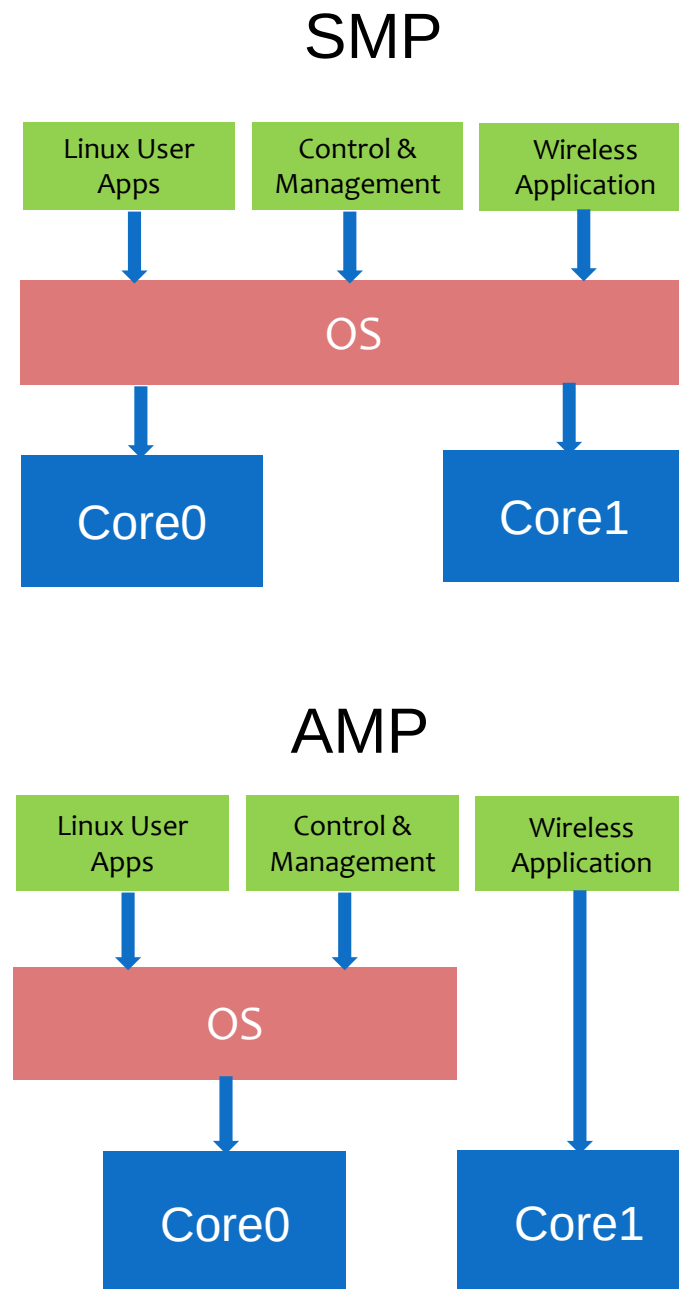
- 多处理器（Multiprocessor）系统和多核处理器（Multi-core Processor）系统的引入对传统的进程模型带来挑战
- 多处理器系统：一台计算机内部包含多个CPU
- 多核处理器也称为片上多处理器（Chip multiprocessors, CMP）：在一颗CPU芯片内集成多个核心。随着大规模集成电路技术的发展，可以将多个CPU核集成到同一芯片内，各个处理器并行执行不同的线程或进程
- 多处理器系统中的处理器可以是多核处理器



Intel Core i7-990X

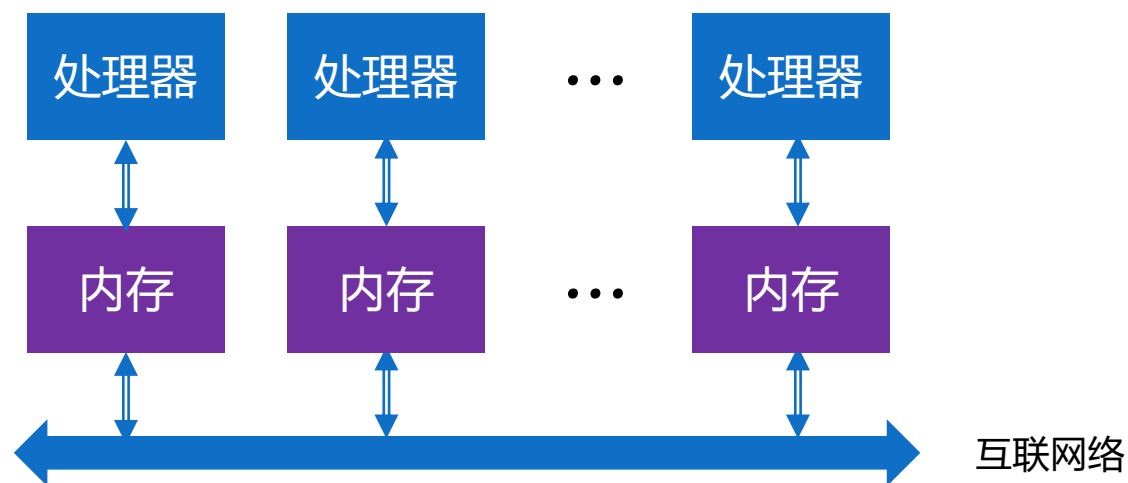
多处理器系统

- 多处理器系统按处理器的关系分为SMP和AMP架构
 - SMP (Symmetric Multiprocessing, 对称多处理)：所有CPU地位对等，共享同一个操作系统实例，任意CPU均可执行操作系统内核代码和用户程序。现代通用CPU的主流模型
 - AMP (Asymmetric Multiprocessing, 非对称多处理)：CPU之间存在主从关系，主CPU负责操作系统及系统控制，从CPU执行特定任务或专用程序。常见于嵌入式系统



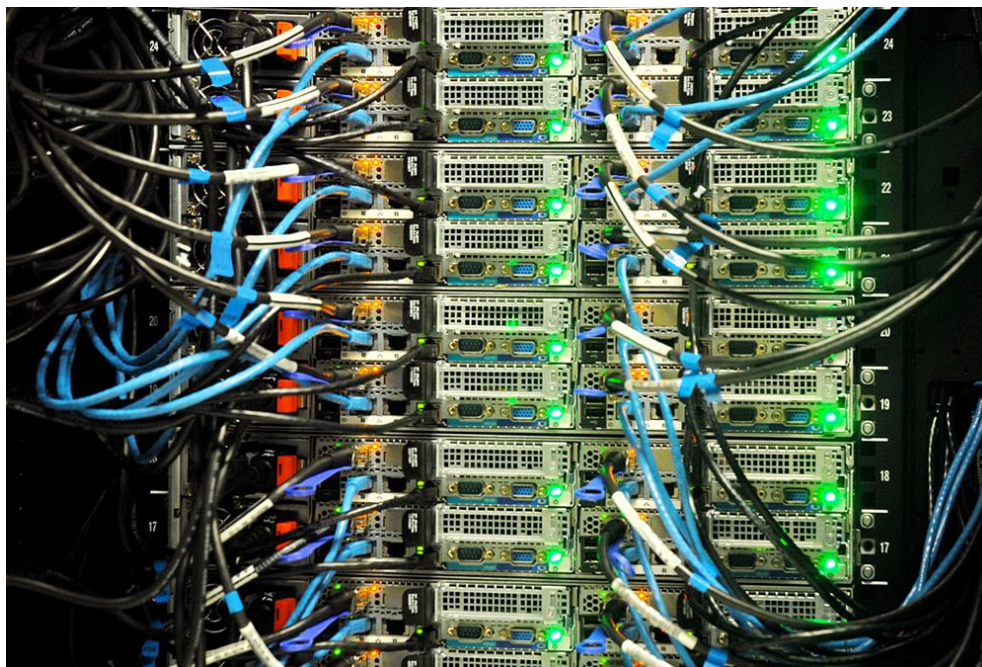
多处理器系统

- 多处理器系统按内存组织分为共享内存架构（包括UMA和NUMA）和非共享内存架构
- 非共享内存架构每个处理器都有一个只有它自己才能访问的局部内存，处理器之间的通信通过消息传递进行



多处理器系统

- 典型的非共享内存架构是集群系统
- 集群系统 (cluster)：每个处理部件都是一个独立的计算机，通过高速网络互连，构成廉价的高性能计算机系统

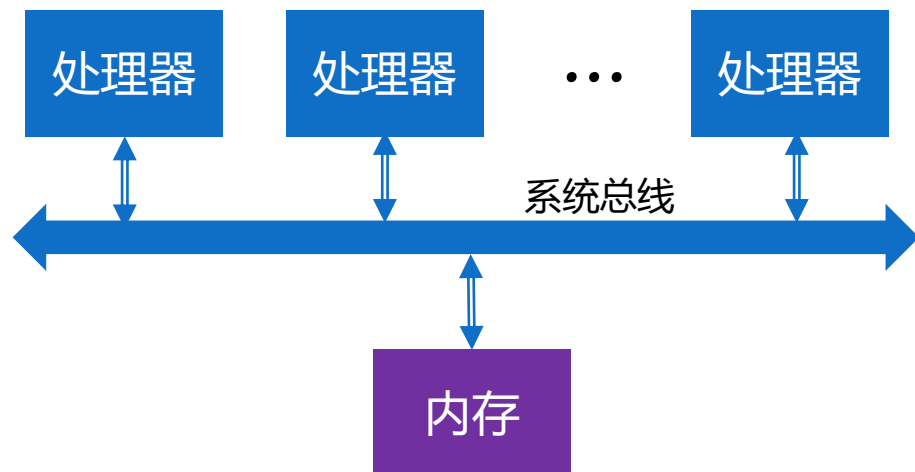


多处理机系统

- 共享内存架构多个处理器共享公用的内存，每个处理器都访问保存在共享内存中的程序和数据，处理器之间可以通过共享内存相互通信
- 共享内存架构多处理机系统进一步分UMA和NUMA结构

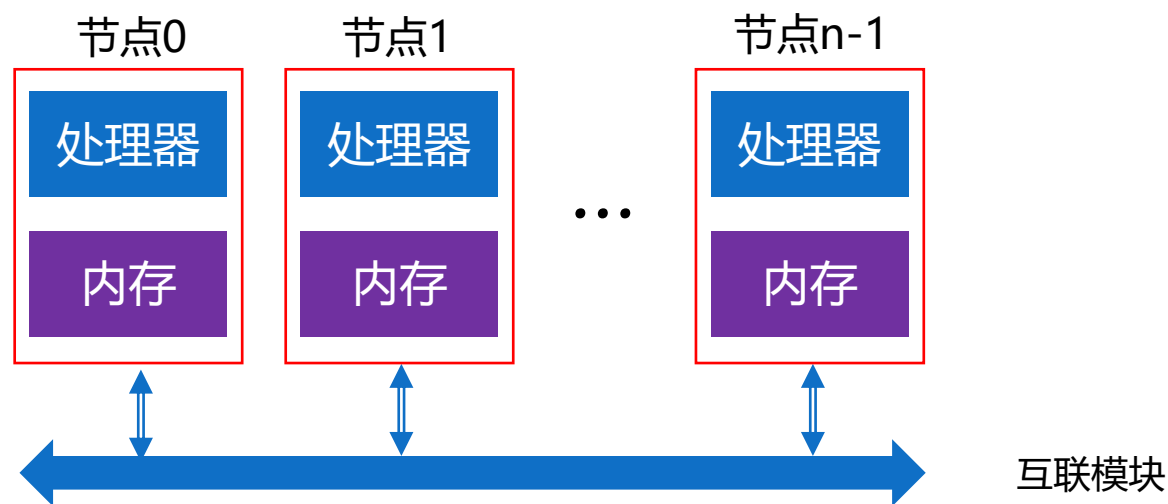
UMA系统

- UMA (Uniform Memory Access, 统一内存访问)
- 所有CPU共享一块物理内存，每个CPU访问任意内存地址的访问延迟相同，内存由统一的内存控制器管理
- 优点：结构简单，编程模型容易理解
- 缺点：可扩展性差，CPU数量较多时总线成为瓶颈



NUMA系统

- NUMA (Non-Uniform Memory Access, 非统一内存访问) 系统由多个节点构成, 每个节点包括CPU (或 CPU 组) 和独立的局部内存, 节点间通过互联模块连接
- 所有内存仍然构成统一地址空间, 可以被所有的处理机访问。访问本地内存快, 访问远程内存慢
- 优点: 扩展性好, 每个节点都有自己的内存带宽, 适合大规模服务器
- 缺点: 数据放置影响性能, 需要操作系统进行NUMA感知调度

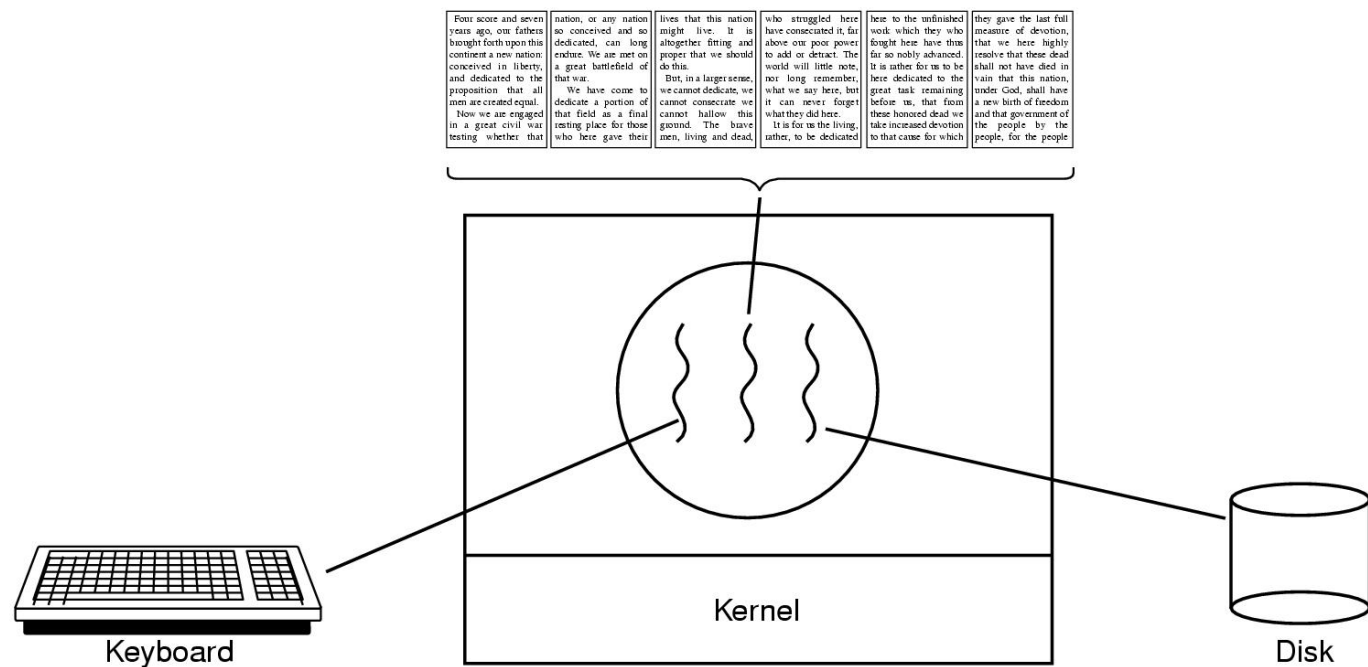


引入线程的动机2

- 在传统的操作系统中，进程是单线程的，单线程意味着进程内部必须是顺序执行，不能并发——即在一个时刻只能运行在一个处理器上，因此不能充分利用多处理器和多核处理器计算机的优势

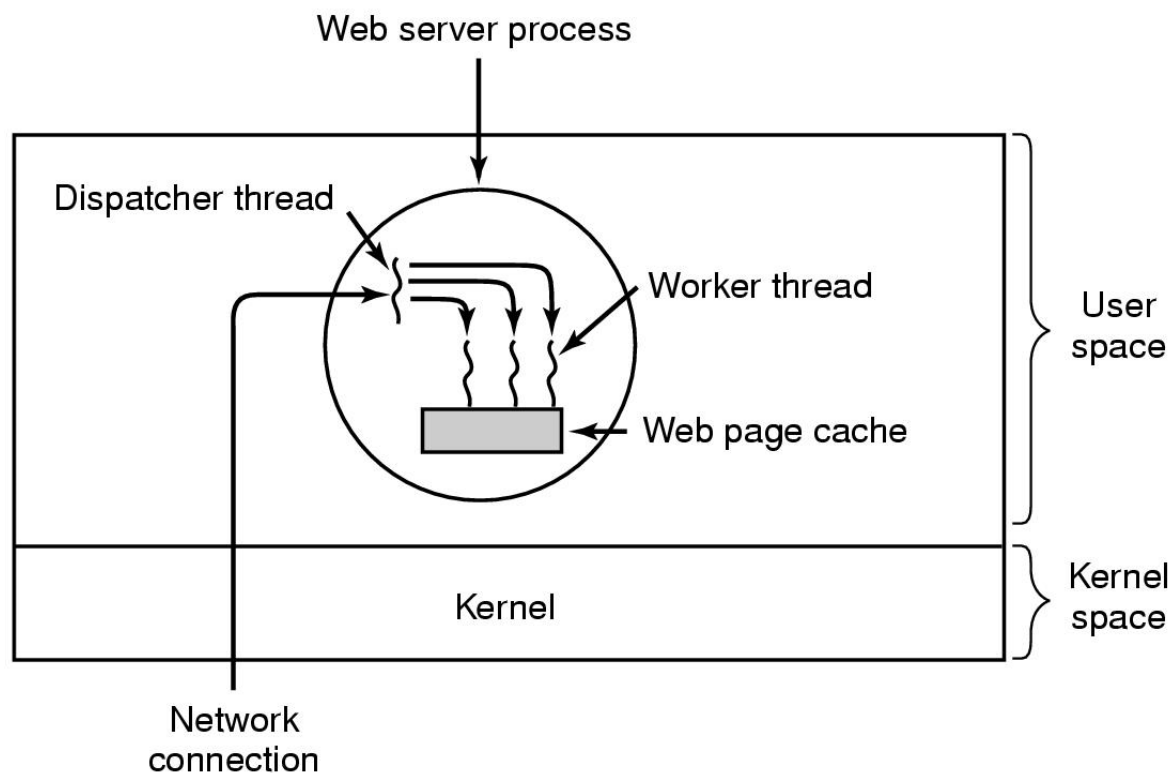
引入线程的动机3

- 现实中有很多需要并发处理的任务，如字处理程序、Web服务器
- 例：字处理程序



引入线程的动机3

- 现实中有很多需要并发处理的任务，如字处理程序、Web服务器
- 例：Web服务器



引入线程的动机

- **解决问题的思路**：将进程的两个基本属性分开，由操作系统分别处理——引入**线程**概念
- 在支持线程的操作系统中，进程只作为资源分配单位，而线程则作为CPU调度单位

日常生活中线程的例子

■ 工程施工

- 整个工程是一个**进程**，下设有许多工程队，如瓦工队、木工队、水电工队等，每个工程队作为一个**线程**
- 进程负责采购资源和工程管理（资源分配和保护）
- 多个工程队可以按进度齐头并进同时工作，以加快工程进度（多线程并行/并发执行）

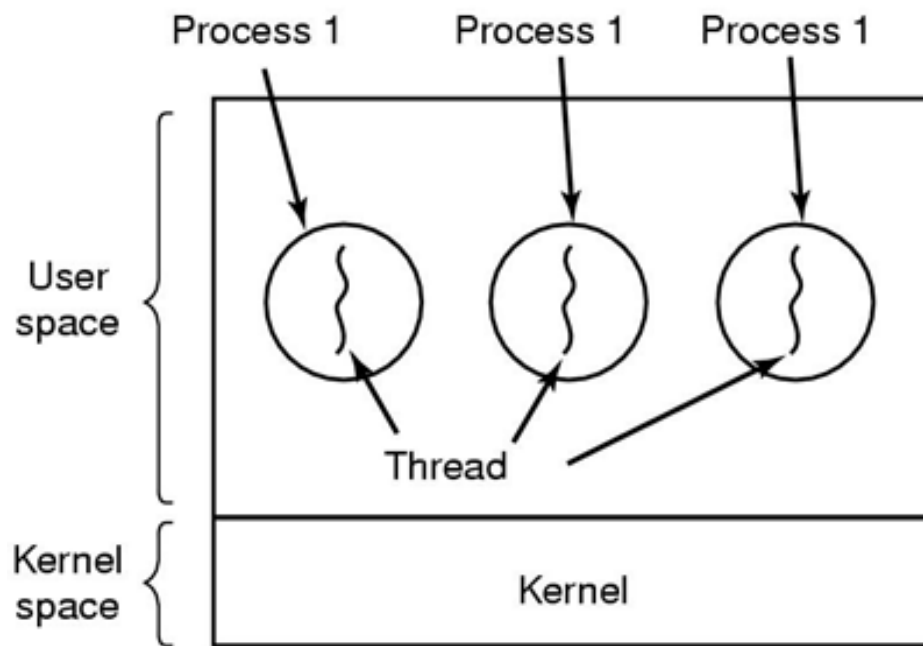


线程概念

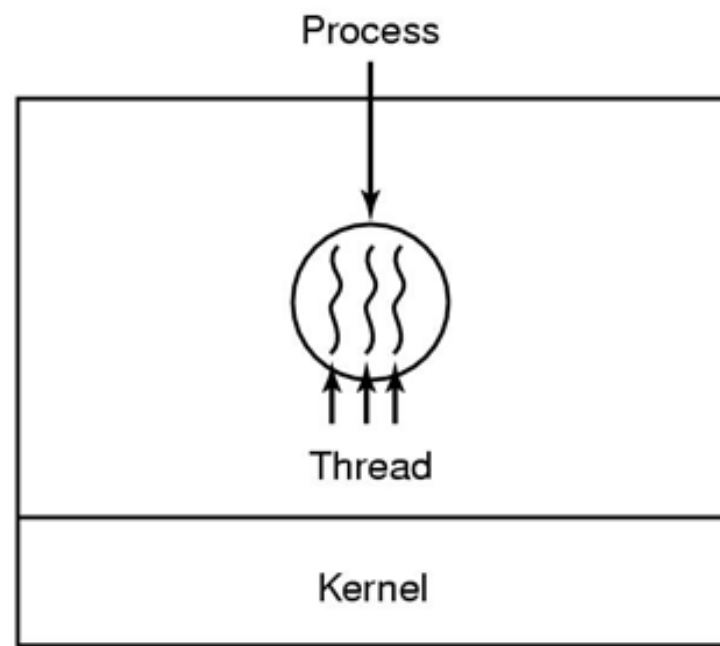
- **线程 (thread)** 是进程中能够独立执行的实体（控制流），是处理器调度的基本单位
- 线程是进程的组成部分，每个进程内允许包含多个并发执行的实体（控制流），这就是**多线程**
- 同一进程中的所有线程共享进程获得的主存空间和资源

多线程

- 多线程 (multithreading) 这一术语用来描述在同一个进程中允许多个线程的情形



单线程



多线程

多线程环境下的进程

- 在多线程环境下，**进程**是操作系统中除处理器外进行资源分配和保护的基本单位
- 进程拥有独立的虚拟地址空间，容纳进程映像（进程所关联的程序的代码与数据），并以进程为单位对各种资源实施保护，例如受保护地访问文件、外部设备及其他进程（进程间通信）
- 由于处理机调度的基本单位是线程而不是进程，因此在创建一个进程时，同时要为该进程创建一个线程（称为**主线程**），否则该进程无法被调度执行

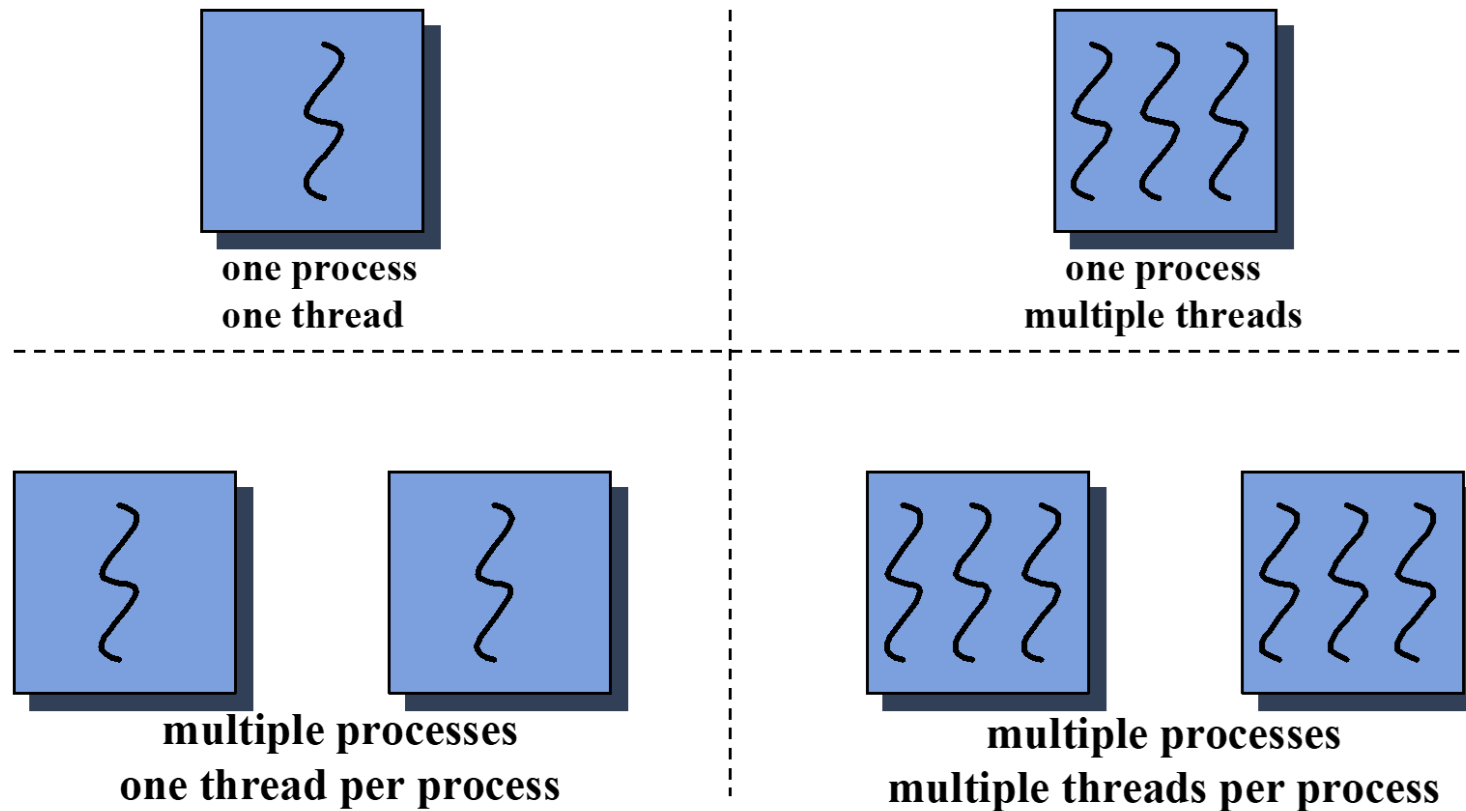
线程模型

- 线程概念试图实现的是共享一组资源的多个线程的执行能力，以便这些线程可以为完成某一任务而共同工作
- 线程共享进程的地址空间和资源

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

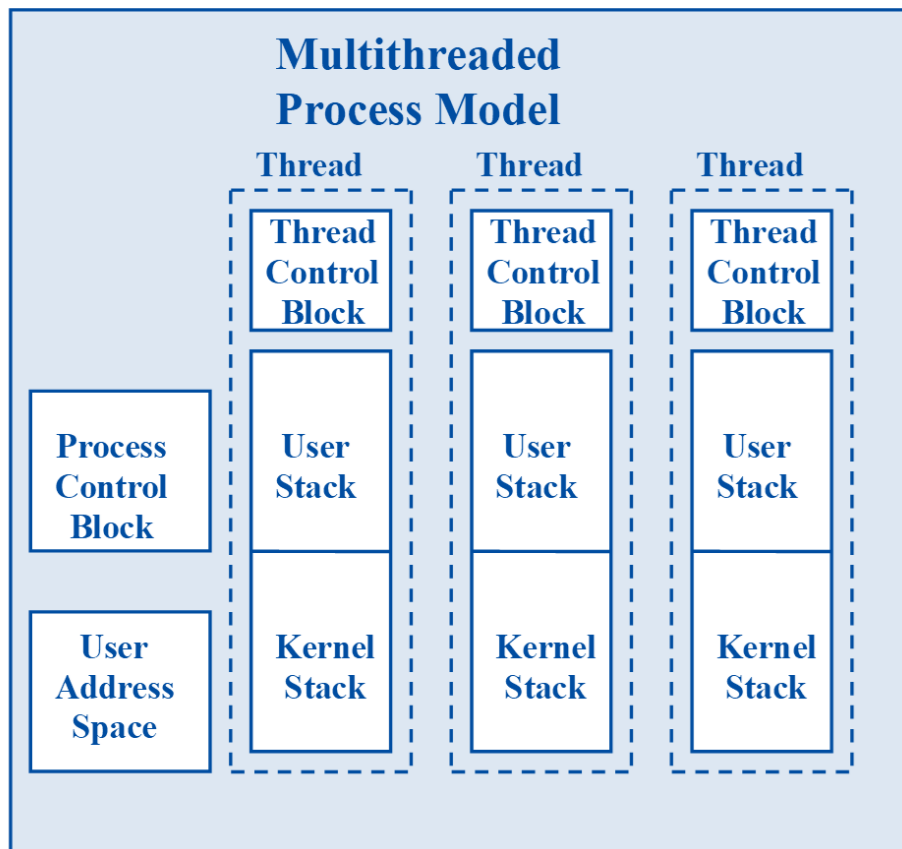
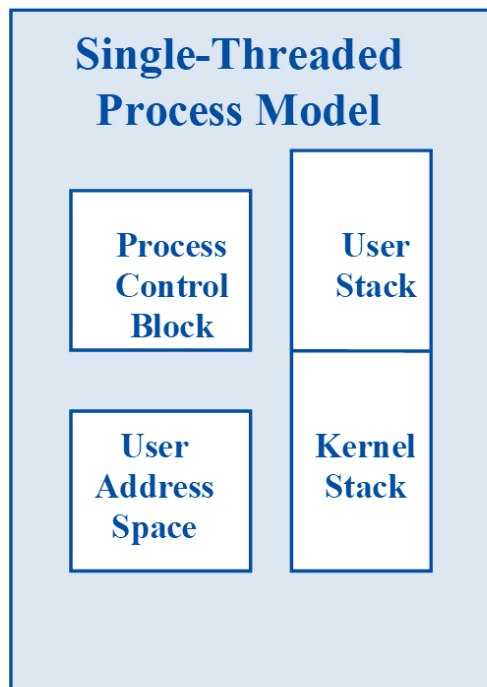
线程模型

不同操作系统中进程和线程的表现形式



线程模型

■ 多线程进程模型与单线程进程模型

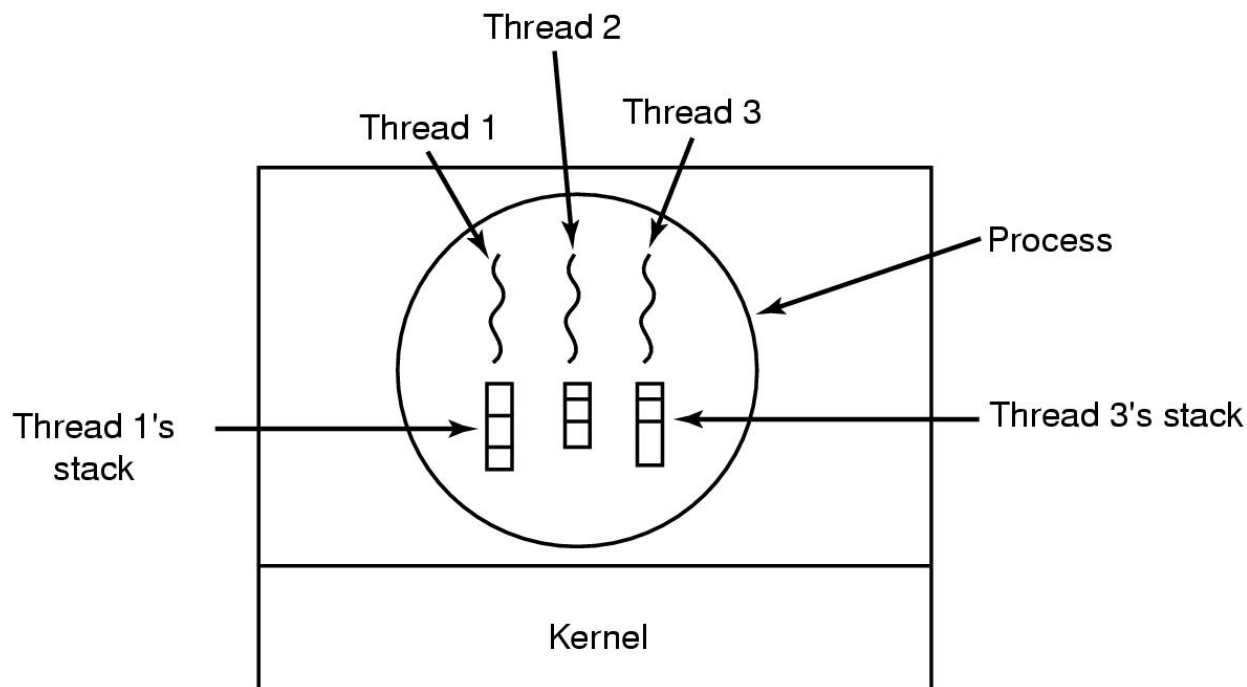


线程的属性

- 执行状态（状态转换）
- 上下文
- 执行栈（包括内核栈和用户栈）
- 可存取所在进程的内存和其他资源
- 可以创建、撤消另一个线程

线程的栈

- 每个线程有自己的栈——函数调用时传递参数，保存返回地址，存放局部变量



线程控制块

- **线程控制块**（TCB, Thread Control Block）是标志线程存在的数据结构，其中包含系统对线程进行管理所需要的全部信息
- TCB中的内容一般较少，因为有关资源分配等多数信息已记录于所属进程的PCB中

线程控制块

- TCB中的主要信息包括：线程标识、线程状态、调度参数、CPU现场信息。其中，CPU现场信息主要包括通用寄存器、指令计数器PC以及用户栈指针。OS内核支持的线程的TCB中还包含系统栈指针
- TCB可能属于操作系统空间，也可能属于用户进程空间，这取决于线程的实现方式

线程的状态

- 线程状态有运行、就绪和阻塞，线程的状态转换类似于进程
- 挂起状态对线程没有意义，如果进程挂起后被对换出主存，则它的所有线程因共享了进程的地址空间，也必须全部对换出去，所以挂起状态是进程级状态，不作为线程级状态。类似地，进程的终止会导致所有线程的终止

进程和线程的比较

- **地址空间和其他资源**：进程间相互独立，同一进程的各线程间共享，某进程内的线程在其他进程不可见
- **通信**：进程间通信采用IPC机制，线程间可以直接读写进程数据段（如全局变量）来进行通信（需要进程同步和互斥手段的辅助，以保证数据的一致性）
- **调度**：线程上下文切换比进程上下文切换要快得多

引入线程的优点

- 减小并发执行的时间和空间开销（线程的创建、退出和调度），因此容许在系统中建立更多的线程来提高并发程度
- 创建一个新线程花费时间少（结束亦如此）
- 两个线程的切换花费时间少（如果机器设有保存/恢复所有寄存器指令，则整个切换过程用几条指令即可完成）
- 因为同一进程内的线程共享内存和文件，因此它们之间相互通信无须调用内核
- 适合多处理机系统

引入线程的缺点

- 在程序设计模型中引入了某种程度的复杂性
- 一个线程的崩溃可能影响到整个进程的稳定性
- 线程能够提高的总性能有限，并且线程多了之后，线程本身的调度也更加复杂，需要消耗较多的CPU资源

下述对线程的描述，正确的是

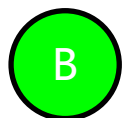
- ☒ A 线程包含CPU现场，可以独立执行
- ☐ B 每个线程有自己独立的地址空间
- ☐ C 进程只能包含一个线程
- ☐ D 线程之间的通信必须使用OS系统调用

提交

UMA和NUMA哪个可扩展性更强？



UMA



NUMA

提交

大纲

01

线程的基本概念

02

线程的实现机制

03

Linux和Windows的线程

线程的实现机制

- 用户级线程
- 内核级线程
- 混合实现

用户级线程

- 用户级线程(User Level Thread, ULT)
- 线程管理工作由应用程序完成，在用户空间内实现
 - 通过用户空间**线程库（运行时系统）**管理线程。线程库是一个管理ULT的例行程序包，实质上是线程的运行支撑环境
 - 应用程序需通过调用线程库中的例程进行编程，与线程库链接后形成可执行程序，以实现多线程
- 操作系统内核不知道线程的存在
- 线程切换不需要核心态特权
- 线程调度是应用特定的

用户级线程

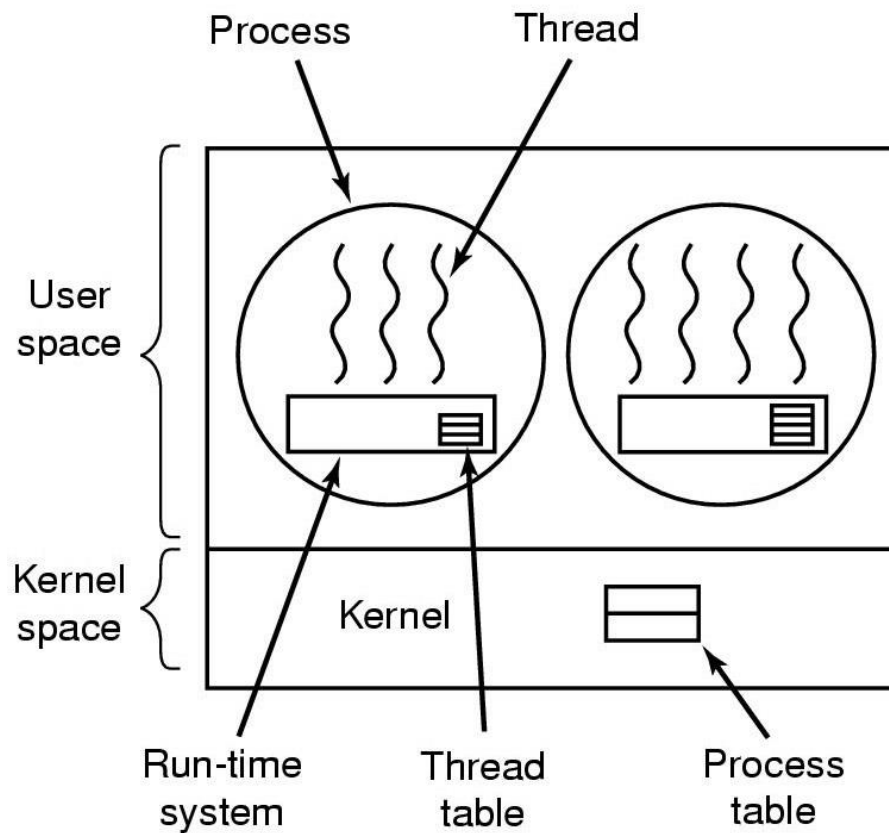
- 线程库（运行时系统）
 - 创建、撤消线程
 - 在线程之间传递消息和数据
 - 调度线程执行
 - 保护和恢复线程上下文
- POSIX的pthread
 - pthread是广泛采用的POSIX多线程编程接口

用户级线程

- 每个进程需要有专用的**线程表 (thread table)**，用来跟踪该进程中线程的状态
- 线程表中的每个表项即线程控制块TCB。每个TCB对应一个线程，记录该线程的属性，如线程的PC、栈、状态和寄存器等
- 线程表由运行时系统（线程库）管理，当线程由运行态转换为就绪态或阻塞态时，在TCB中存放重新运行该线程所需的信息

用户级线程

■ 线程在一个运行时系统之上运行



用户级线程与操作系统内核的关系

- 操作系统内核不知道线程的活动，仍然管理线程所从属的进程的活动——处理器调度的单位是进程而不是线程
- 当线程调用阻塞型系统调用时，整个进程阻塞
- 线程状态是与进程状态独立的

用户级线程与操作系统内核的关系

■ 假设正在执行进程P中的线程T1，可能出现下列情况：

- ① 线程T1发出一个阻塞式系统调用（例如I/O操作），导致控制转到内核，内核启动I/O操作，将进程P置为阻塞态，并切换到另一个进程。按照由线程库所维护的数据结构，线程T1仍处在运行态
- ② 进程P时间片用完，内核从进程P剥夺处理器，把它放入就绪队列，切换到另一个就绪进程。此时，由于进程P中线程T1正在运行，按由线程库维护的数据结构，线程T1仍处于运行态，但是进程P处于就绪态
- ③ 线程T1执行到某处，它需要线程T2的某些操作，于是让线程T1变成阻塞态，而线程T2从就绪态转为运行态，在这一过程中进程P始终处于运行态

用户级线程的优点

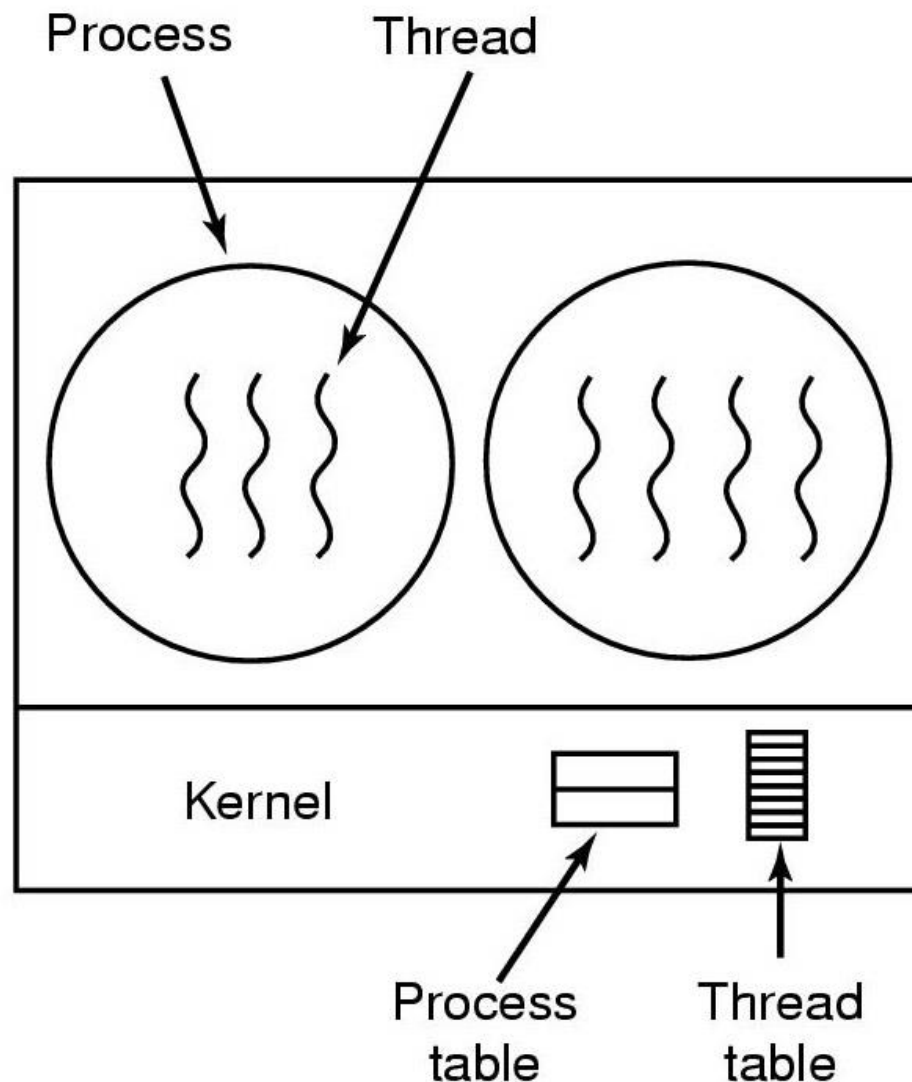
- 线程切换不调用操作系统内核，性能良好
- 调度是应用程序特定的，可针对应用优化。线程库的线程调度算法与操作系统的低级调度算法是无关的
- 无论操作系统内核是否支持线程，ULT都可以运行，只需要线程库即可，许多当代操作系统和程序设计语言均提供了线程库

用户级线程的缺点

- 调度通常采用非抢先式和更简单的规则
- 大多数系统调用是阻塞的，因此内核阻塞进程，故进程中所有线程将被阻塞
- 操作系统内核只将处理器分配给进程，同一进程中的两个线程不能同时运行于两个处理器上

内核级线程

- 线程的管理工作由内核完成
- 在操作系统内核中需要有记录系统中所有线程的线程表，其中包括每个线程的线程控制块(TCB)
- 不需要线程库
- 每个进程中不需要线程表



内核级线程的优点

- 在多处理器上，内核能够同时调度同一进程中的多个线程并行执行
- 阻塞是在线程一级完成。若进程中的一个线程被阻塞，内核能够调度同一进程的其它线程占有处理器运行，也可以运行其它进程中的线程
- 内核自身也可以用多线程技术实现，从而提高系统执行效率

内核级线程的缺点

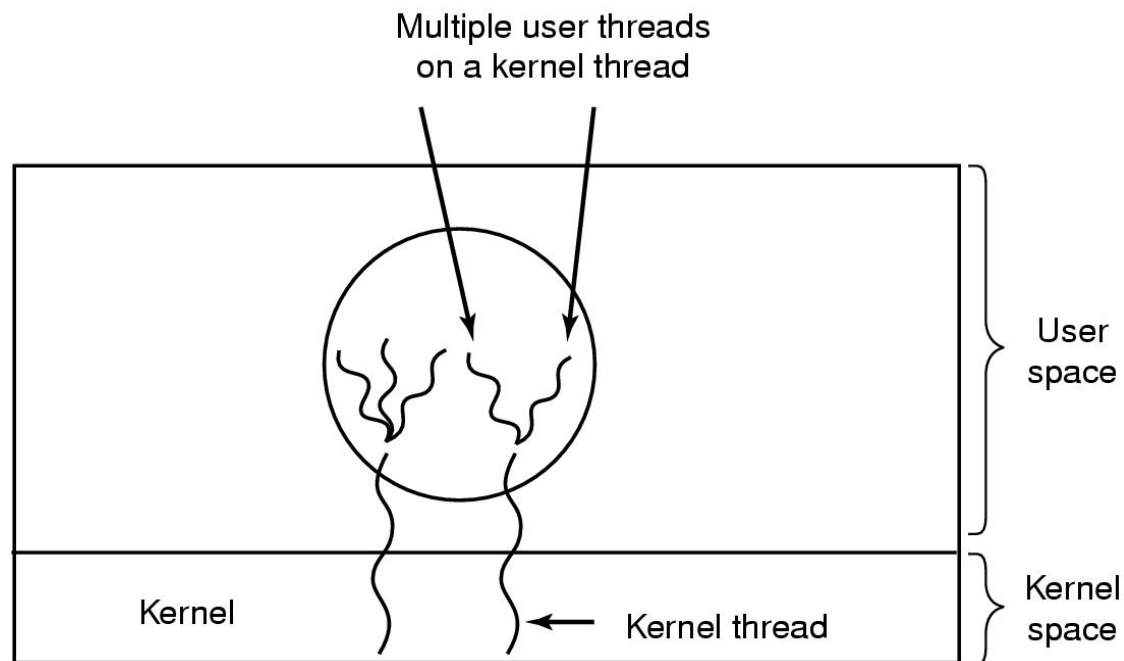
- 线程在用户态运行，调度和管理在内核实现，因此需要频繁进行模式切换，系统开销较大

混合实现

- 使用内核级线程，然后将用户级线程与某些或全部内核级线程多路复用起来
- 内核只认识内核级线程，并对其进行调度
- 某些内核级线程的顶部有若干可以轮流使用的用户级线程集合

混合实现

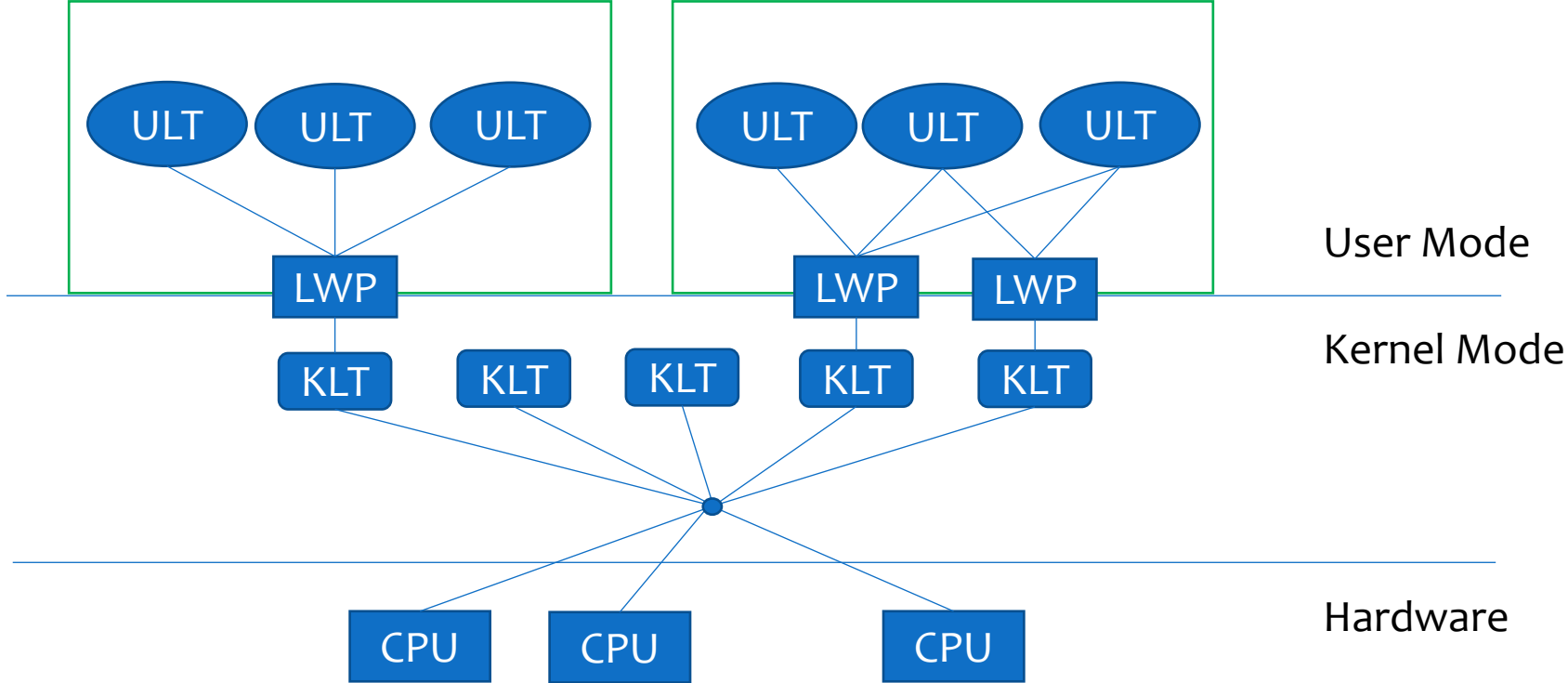
- 在线程的混合实现机制中，线程创建、调度、同步在用户级线程中完成，应用程序的多个ULT将被映射到一些KLT上
- 程序员可以根据情况调节KLT数量，以求系统达到最佳效果



混合实现——Solaris的线程模型

- Solaris实现了三个层次的线程结构（ULT、LWP、KLT），有利于操作系统管理线程，并为应用程序提供了接口API
- LWP（Light Weight Process，轻量级进程）是一种由内核支持的用户线程。它是基于内核线程的高级抽象，因此只有先支持内核线程，才能有LWP。每一个进程有一个或多个LWP，每个LWP由一个内核线程支持，LWP由内核独立调度，能够在多处理器中并发执行
- LWP可以支持多个ULT，其作用是作为ULT和KLT之间的桥梁

混合实现——Solaris的线程模型



混合实现——Solaris的线程模型

- Solaris的LWP层实现了ULT到KLT的多对多映射，灵活、高效
- 实现过于复杂，从Solaris 10之后被放弃

内核级线程指的是

- ☐ A 由内核管理的内核线程
- ☒ B 由内核管理的用户线程
- ☐ C 在内核态下运行的线程
- ☐ D 在用户态下运行的线程

提交

在多线程Web服务器中，如果读文件的唯一途径是常规的阻塞式read()系统调用，你认为Web服务器应该使用用户级线程还是内核级线程？

- ☐ A 用户级线程
- ☒ B 内核级线程

提交

大纲

01

线程的基本概念

02

线程的实现机制

03

Linux和Windows的线程

POSIX线程

- 早期的UNIX并没有线程概念
- POSIX1003.4a的设计者们经过多年努力，制定了一套多线程编程接口标准pthread
- POSIX规范并没有规定线程应该在内核中实现还是在用户空间中实现，只要用户程序严格遵守pthread语法，两种实现都会正确工作
- 当使用内核线程时pthread定义的调用是真正的系统调用；当使用用户线程时pthread定义的调用以用户空间运行时线程库的现实形式

POSIX线程

■ pthread线程函数

Thread call	Description
pthread_create	Create a new thread in the caller's address space
pthread_exit	Terminate the calling thread
pthread_join	Wait for a thread to terminate
pthread_mutex_init	Create a new mutex
pthread_mutex_destroy	Destroy a mutex
pthread_mutex_lock	Lock a mutex
pthread_mutex_unlock	Unlock a mutex
pthread_cond_init	Create a condition variable
pthread_cond_destroy	Destroy a condition variable
pthread_cond_wait	Wait on a condition variable
pthread_cond_signal	Release one thread waiting on a condition variable

POSIX线程

■线程创建

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr,  
void *(*start_routine)(void *), void *arg);
```

■参数

pthread_t *thread: 指向线程标识符的指针

pthread_attr_t *attr: 指向线程属性的指针, 用来设置线程属性

void *(*start_routine)(void *): 线程执行的函数

void *arg: 传递给线程函数的参数

■返回值: 成功, 返回0; 出错, 返回出错号

POSIX线程

■线程中止

```
void pthread_exit(void *value_ptr);
```

■参数

void *value_ptr: 指向返回值的指针

POSIX线程

- 等待一个线程结束，阻塞调用线程

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- 参数

pthread_t thread: 被等待线程的标识符

void **value_ptr: 用户定义的指针，存放被等待线程的返回值

- 返回值：成功，返回0；出错，返回出错号

POSIX线程

■实例pthdtst.c



pthdtst.c

POSIX线程

- 所有使用pthread函数的程序开头

```
#define _REENTRANT
```

```
#include <pthread.h>
```

- 编译时，pthread函数的目标代码必须使用-lpthread选项连接

Linux的线程

- Linux实现线程的机制非常独特
- 从内核的角度，Linux只有进程而没有线程的概念，Linux没有准备特别的调度算法或定义特别的数据结构来表示线程
- 在Linux中，线程仅仅被视为使用某些共享资源的进程，每个线程都拥有自己的task struct，所以在内核看来它就是一个普通的进程，但是该进程与其他进程共享某些资源，例如地址空间

Linux的线程

- Linux实现线程的关键是一个新的系统调用sys_clone(), 该系统调用在UNIX任何其它版本中都不存在
- 对应的封装库函数:

```
int clone(  
    int (*fn) (void *arg), //pointer to function  
    void *child_stack, //pointer to a new stack  
    int flags, //various flags  
    void *arg //argument passed to function  
)
```


Linux的线程

- 根据flags参数的不同，新线程要么创建于当前进程里，要么创建于新进程里
- 如果新线程创建于当前进程里，那么它就与现有线程共享地址空间
- Linux下pthread的pthread_create通过clone系统调用实现
- 现代Linux的fork也是通过clone系统调用实现

Linux的线程

■ clone()的flags参数

Flag	Meaning when set	Meaning when cleared
CLONE_VM	Create a new thread	Create a new process
CLONE_FS	Share umask, root, and working dirs	Do not share them
CLONE_FILES	Share the file descriptors	Copy the file descriptors
CLONE_SIGHAND	Share the signal handler table	Copy the table
CLONE_PID	New thread gets old PID	New thread gets own PID

Linux的线程

■ 实例clonetest.c



clonetest.c

Linux的进程与线程

- Linux的fork也是通过clone系统调用实现，fork()只是clone()的一个封装
- fork调用clone时，flags不设置CLONE_VM、CLONE_FS等标志，即为创建进程；设置SIGCHLD，使得在进程终止或者停止时，将SIGCHLD信号发送给其父进程

Windows的线程

- Windows支持内核级线程
- 在Windows中，进程是惰性的，即进程什么也不执行，它只是拥有虚拟地址空间用来存放应用程序的EXE文件所需要的代码和数据。 EXE文件所需要的DLL也加载到进程的地址空间中
- 除了地址空间外，进程还拥有其他资源

Windows的线程

- 由于Windows的进程是惰性的，它必须至少拥有一个线程来负责执行包含在进程地址空间中的代码
- 单个进程可以拥有多个线程，它们都在进程的地址空间中并发地执行代码
- 每个线程拥有自己的CPU寄存器组和栈
- 当Windows进程被创建时，它的第一个线程(称为主线程)由系统自动创建。主线程然后可以创建更多新的线程，新的线程也能更多新的线程

Windows的线程

- Windows线程由执行体线程块ETHREAD表示，即线程对象，其中包含内核线程块KTHREAD，即线程控制块TCB
- ETHREAD

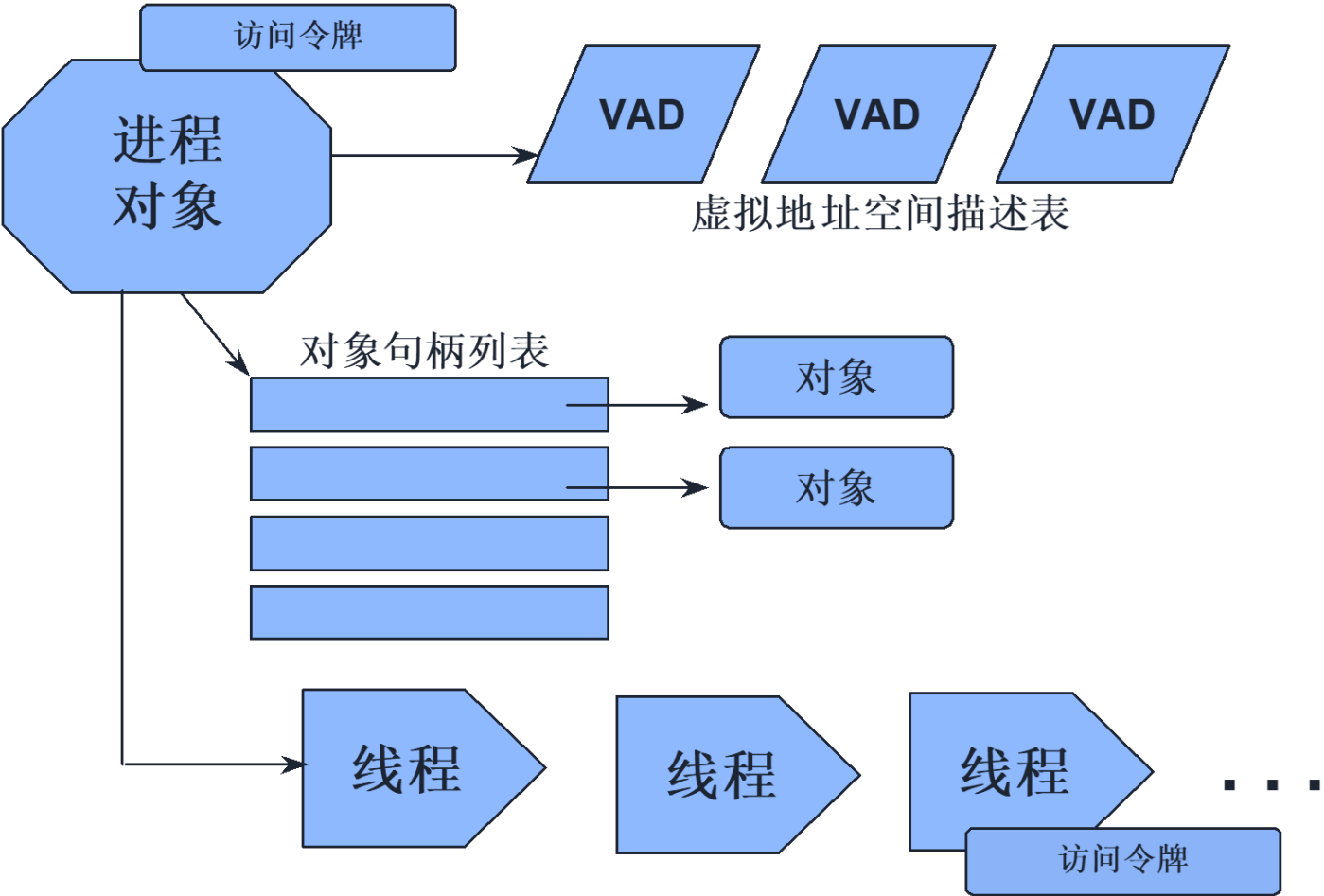
```
kd> dt _ETHREAD
ntdll!_ETHREAD
+0x000 Tcb                : _KTHREAD
+0x1b8 CreateTime         : _LARGE_INTEGER
+0x1c0 ExitTime           : _LARGE_INTEGER
+0x1c0 LpcReplyChain       : _LIST_ENTRY
+0x1c0 KeyedWaitChain     : _LIST_ENTRY
+0x1c8 ExitStatus         : Int4B
+0x1c8 OfsChain           : Ptr32 Void
+0x1cc PostBlockList      : _LIST_ENTRY
.....
```

Windows的线程

■ KTHREAD

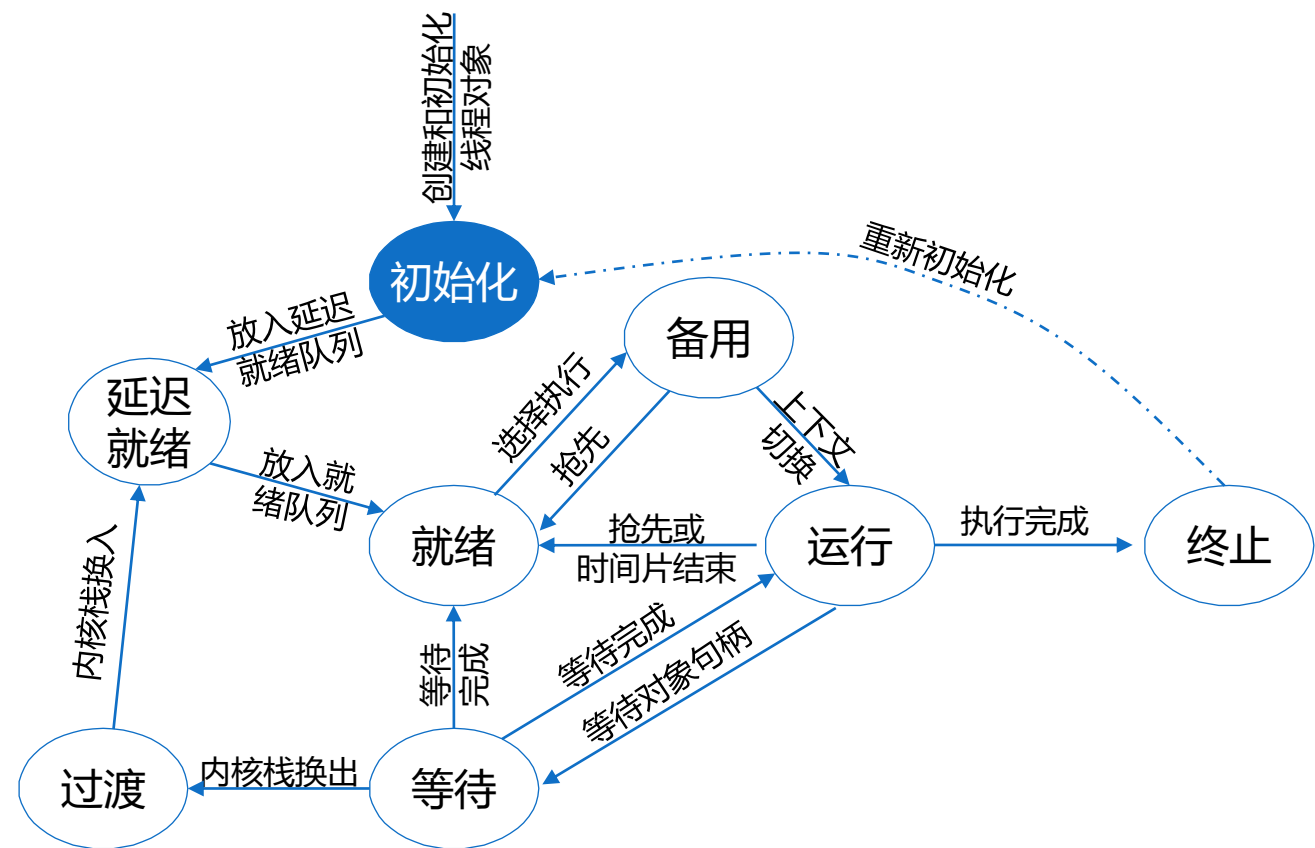
```
kd> dt _KTHREAD
ntdll!_KTHREAD
+0x000 Header           : _DISPATCHER_HEADER
+0x010 MutantListHead   : _LIST_ENTRY
+0x018 InitialStack     : Ptr32 Void
+0x01c StackLimit       : Ptr32 Void
+0x020 KernelStack      : Ptr32 Void
+0x024 ThreadLock       : Uint4B
+0x028 ApcState          : _KAPC_STATE
+0x028 ApcStateFill     : [23] UChar
+0x03f ApcQueueable     : UChar
+0x040 NextProcessor     : UChar
.....
```


Windows的进程与线程的关系



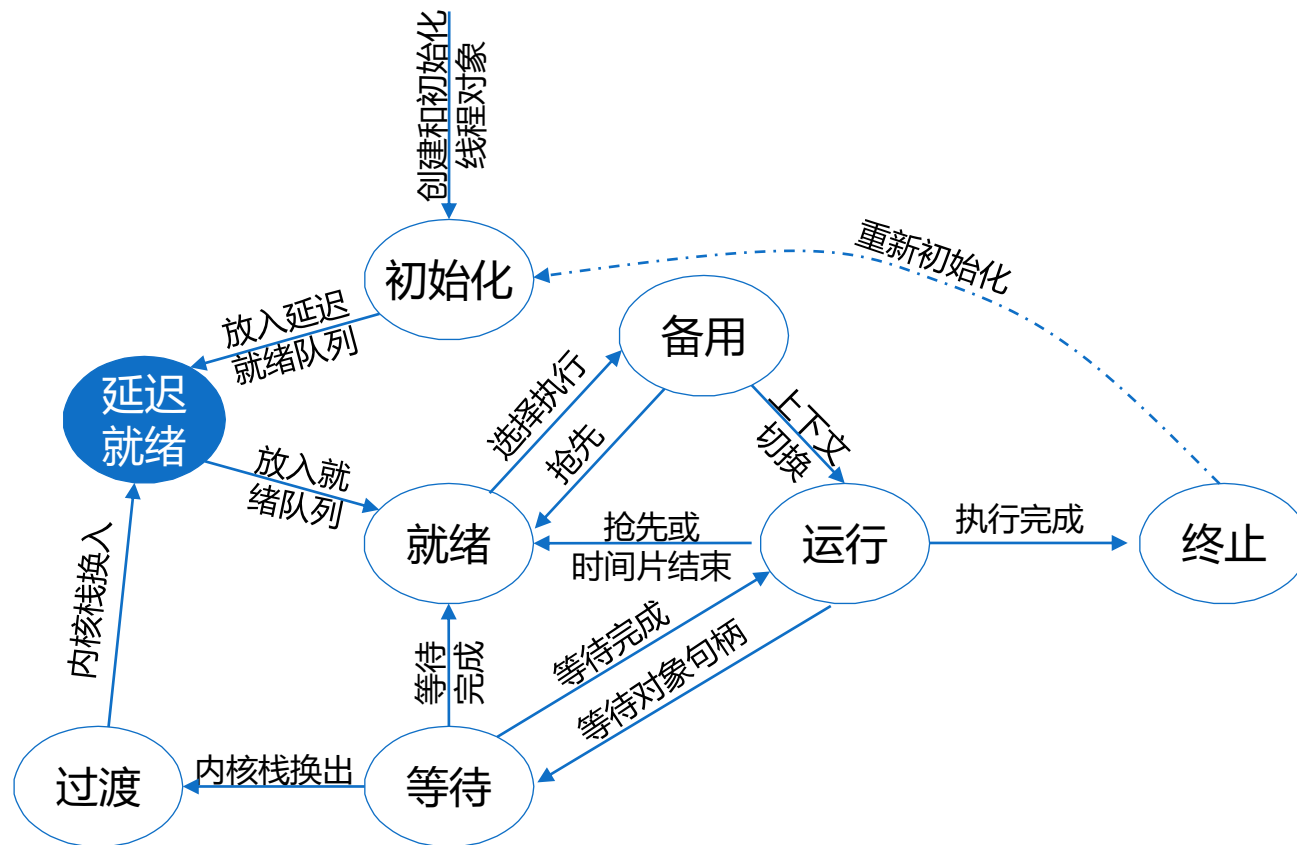
Windows线程的状态

■初始化(Initialized): 线程创建过程中的线程状态



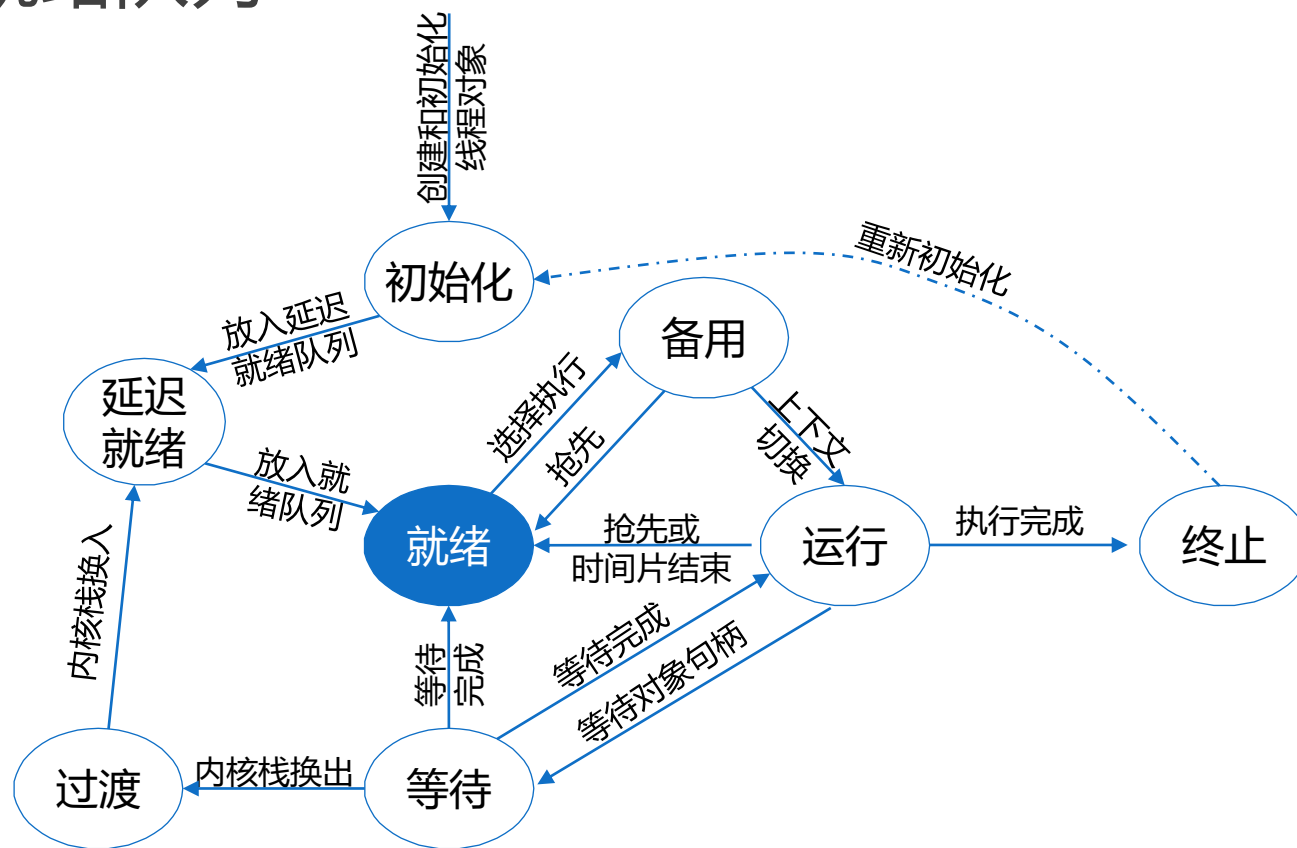
Windows线程的状态

- **延迟就绪(Deferred Ready)**: 线程已经满足运行条件但调度器暂时还没把它放入就绪队列。延迟就绪是一种优化状态，目的是减少锁竞争，提高调度效率



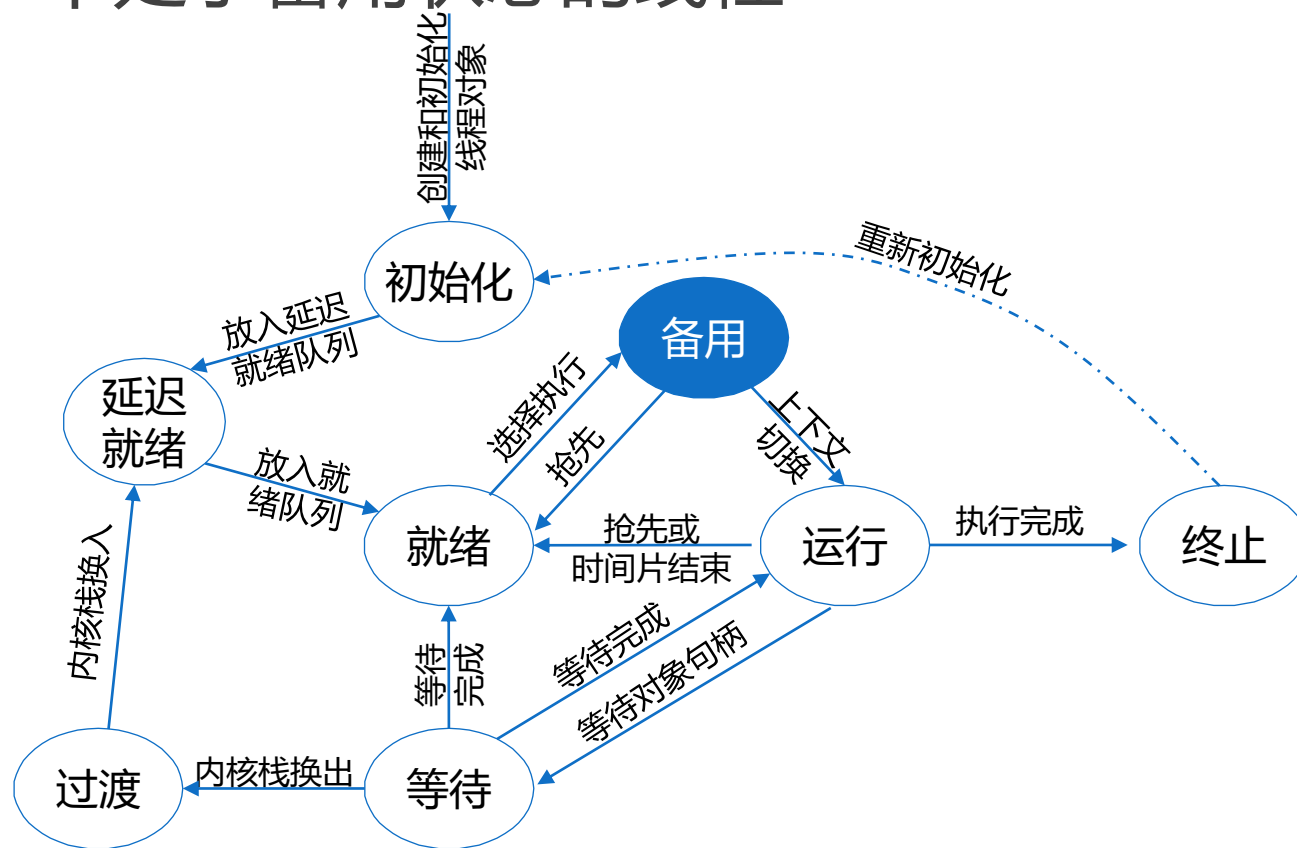
Windows线程的状态

- **就绪(Ready)**: 线程已获得除处理机外的所需资源, 等待执行。每个CPU有自己的就绪队列



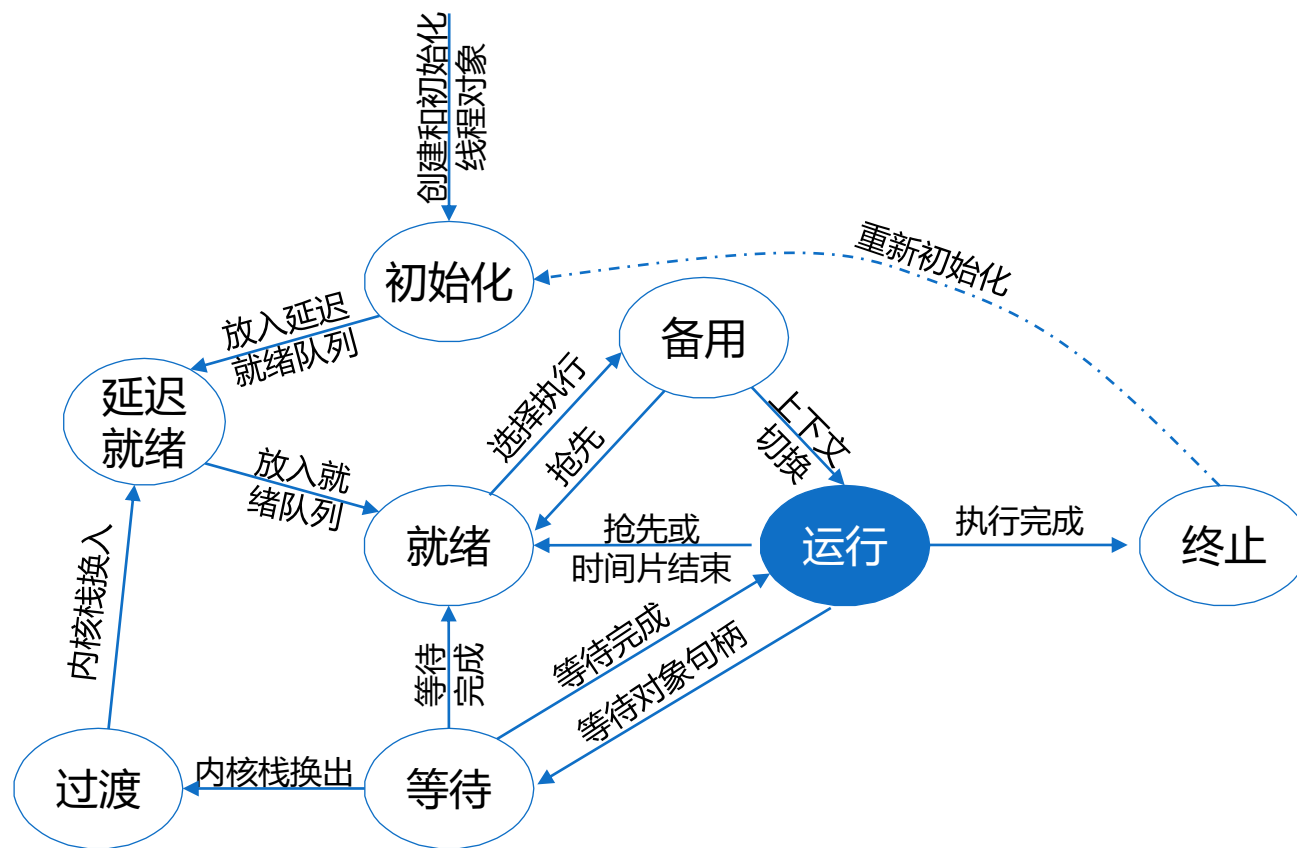
Windows线程的状态

- **备用(Standby)**: 已选择好处理器, 正等待上下文切换。系统中每个处理器上只能有一个处于备用状态的线程

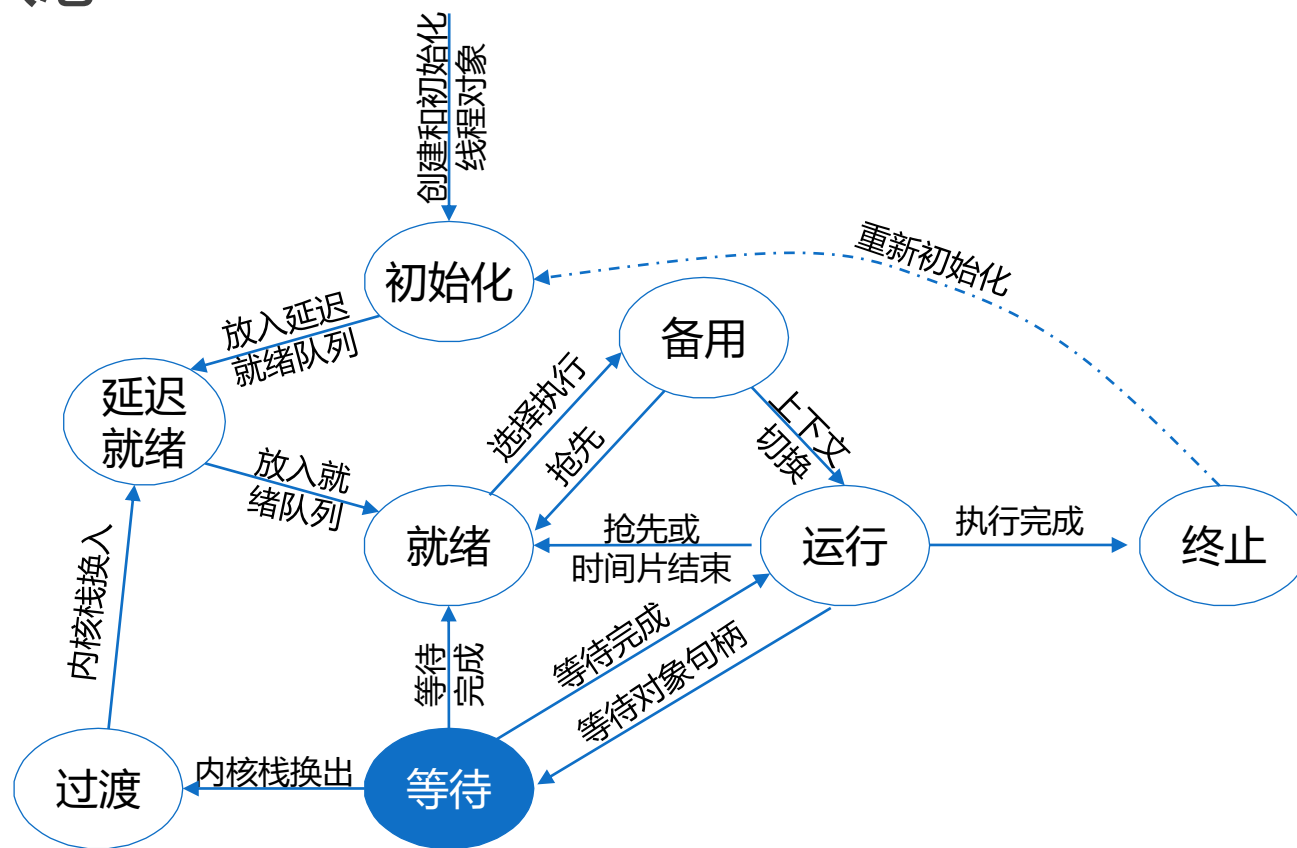


Windows线程的状态

■运行(Running): 完成上下文切换, 线程进入运行状态

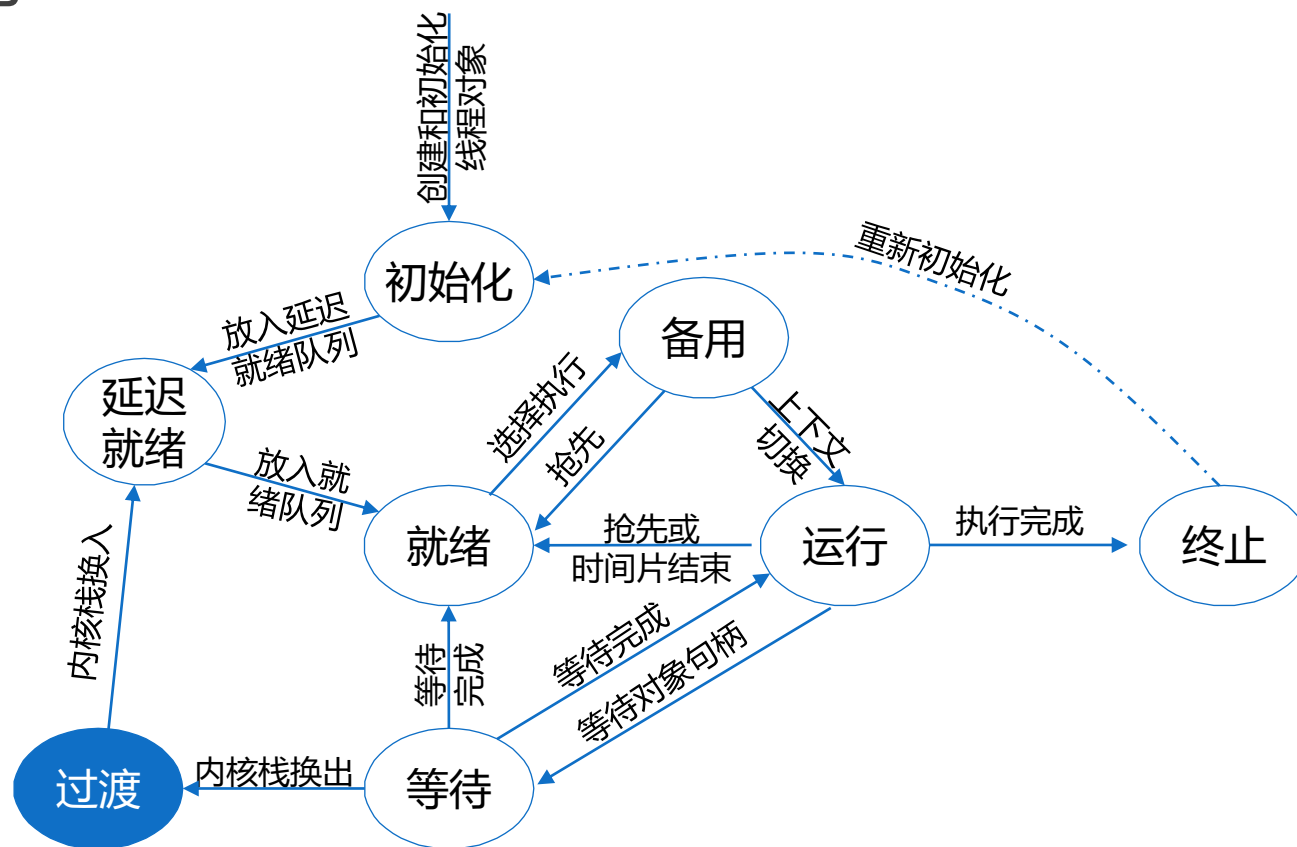


Windows线程的状态



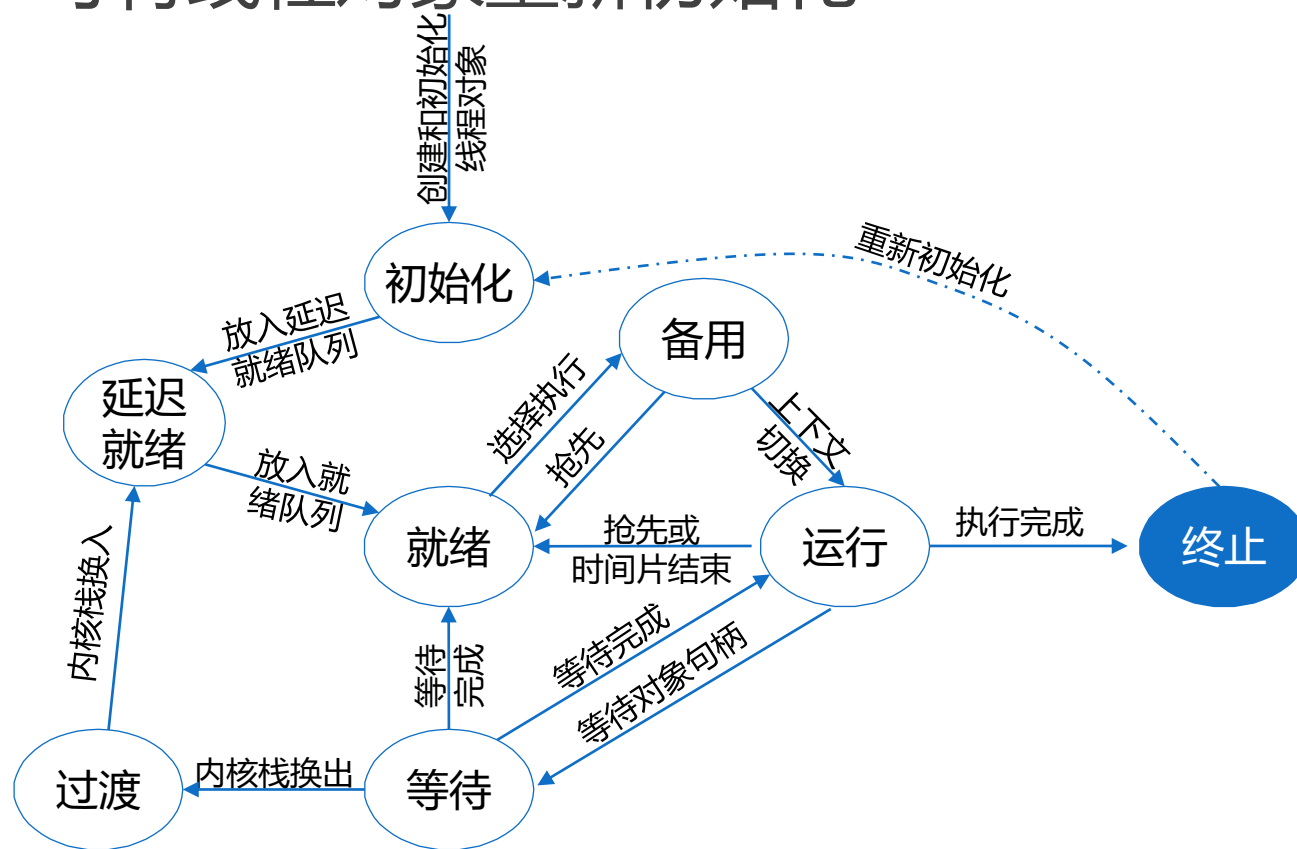
Windows线程的状态

- **过渡(Transition)**: 线程在准备执行而其内存被换出不在工作集中，线程进入过渡状态



Windows线程的状态

- **终止(Terminated)**: 线程执行完就进入终止状态。如执行体有一指向线程对象的指针，可将线程对象重新初始化



Windows线程的状态

- Windows的线程状态分成两类：队列驱动的状态和瞬时状态
- 队列驱动的状态包括就绪态和等待态，线程之所以进入队列，是因为存在竞争。就绪队列中的线程竞争CPU，在某个等待对象的队列中排队的线程竞争锁或者事件
- 瞬时状态包括Running（正在CPU上执行，无需排入任何队列）、Deferred Ready（临时挂在一个延迟链表中）、Standby（已被选中即将运行，通常只是CPU的一个指针，而不是队列）、Transition（等内存资源，依赖内存管理结构，而不是统一队列）

Windows线程的状态

- Windows线程状态及其转换是对经典三状态（就绪、运行、阻塞）模型的工程化细化和实现层扩展
- Windows线程状态划分更细：三态模型只关心进程能不能运行，但Windows还要关心下一个运行的是哪个线程？线程是否可以进入调度队列？这是在多核和高并发环境下操作系统必须要考虑的问题

Windows线程的状态

经典进程三状态	Windows线程状态	说明
Ready	Deferred Ready	尚未入进入就绪队列
Ready	Ready	标准就绪
Ready	Standby	即将运行
Running	Running	正在执行
Waiting	Waiting	等事件
Waiting	Transition	等内存

与Windows线程有关的API

■ CreateThread()

- 在调用者的进程地址空间上创建一个线程，以执行指定的函数；返回值为所创建线程的句柄

■ ExitThread()

- 用于结束本线程

■ SuspendThread()

- 用于挂起指定的线程

■ ResumeThread()

- 递减指定线程的挂起计数，挂起计数为0时，线程恢复执行

与Windows线程有关的API

■CreateThread()

```
HANDLE WINAPI CreateThread(  
    LPSECURITY_ATTRIBUTES    lpThreadAttributes,  
    SIZE_T                   dwStackSize,  
    LPTHREAD_START_ROUTINE   lpStartAddress,  
    LPVOID                   lpParameter,  
    DWORD                    dwCreationFlags,  
    LPDWORD                  lpThreadId  
);
```

lpThreadAttributes 线程内核对象的安全属性，一般传入NULL表示使用默认设置

dwStackSize 线程栈空间大小。传入0表示使用默认大小（1MB）

lpStartAddress 新线程所执行的线程函数地址，多个线程可以使用同一个函数地址

lpParameter 传给线程函数的参数

dwCreationFlags 指定额外的标志来控制线程的创建，为0表示线程创建之后立即就可以进行调度，如果为CREATE_SUSPENDED则表示线程创建后暂停运行，这样它就无法调度，直到调用ResumeThread()

lpThreadId 返回线程的ID号，传入NULL表示不需要返回该线程ID号

Windows的线程

■ 实例thread.c



thread.c

操作系统，2026

谢谢

OPERATING SYSTEM